🚀 NEO AI — Production Deploy Files for TRAININGBOT2

📁 Exact File Placement

Copy each file to the **exact** location shown below in your `TRAININGBOT2` folder:

```
C:\Users\fundi\OneDrive\Documents\TRAININGBOT2
│
├── 📄 wsgi.py                          ← ROOT (new file)
├── 📄 Dockerfile                       ← ROOT (new file)
├── 📄 requirements.txt                 ← ROOT (new file)
├── 📄 docker-compose.yml               ← ROOT (new file)
├── 📄 ecosystem.config.js              ← ROOT (new file)
├── 📄 setup.sh                         ← ROOT (new file)
├── 📄 nginx.conf                       ← ROOT (new file, optional)
│
├── 📁 .github
│   └── 📁 workflows
│       └── 📄 deploy.yml               ← .github\workflows\ (new file)
│
├── 📄 launcher.py                      ← ROOT (your existing file)
├── 📄 launcher_integrated.py           ← ROOT (your existing file)
├── 📄 neo_chat.html                    ← ROOT (your existing file)
├── 📄 .env                             ← ROOT (your existing file)
│
├── 📁 core                             ← EXISTING (your modules)
│   ├── 📄 local_llm.py
│   ├── 📄 llm_router.py
│   ├── 📄 orchestrator.py
│   ├── 📁 plugins
│   └── ...
│
├── 📁 data                             ← EXISTING
│   └── 📁 vector_store
│
└── 📁 logs                             ← EXISTING
```

🎯 What Each File Does

File	Location	Purpose	Required?
wsgi.py	Root	Gunicorn entry point — initializes Neo then exposes Flask app	✅ Yes
Dockerfile	Root	Multi-stage Docker build for cloud deployment	✅ Yes
requirements.txt	Root	Production Python dependencies	✅ Yes

Generated by Kimi.ai

Table 1 – continued

File	Location	Purpose	Required?
docker-compose.yml	Root	Runs Neo API + Ollama + Nginx together	⚠️ Optional
ecosystem.config.js	Root	PM2 process manager (auto-restart, logs)	⚠️ Optional
setup.sh	Root	One-click Ubuntu/Oracle Cloud setup	⚠️ Optional
nginx.conf	Root	Reverse proxy with SSL and rate limiting	⚠️ Optional
deploy.yml	`.github\workflows\`	GitHub Actions CI/CD pipeline	⚠️ Optional

⚡ Quick Deploy (Render — Easiest)

1. **Copy files** to TRAININGBOT2 root (see above)
2. **Push to GitHub:**

```
git add .
git commit -m "Production deploy ready"
git push origin main
```

3. **Go to render.com**
4. Click **"New Web Service"** → Connect your GitHub repo
5. Render auto-detects the `Dockerfile` and deploys
6. Done! Your API is live at `https://neo-ai-xxx.onrender.com`

🖥️ Quick Deploy (Oracle Cloud — Always-On)

1. **SSH into your free Oracle VM:**

```
ssh ubuntu@your-oracle-ip
```

2. **Upload `setup.sh` and run:**

```
chmod +x setup.sh
./setup.sh
```

3. **Copy your `.env` file:**

```
scp .env ubuntu@your-oracle-ip:~/TRAININGBOT2/
```

4. **Neo is live 24/7!**

🔧 Before You Deploy — Update These

1. `setup.sh` — Line 8

```
REPO_URL="https://github.com/yourusername/TRAININGBOT2.git"
# Change "yourusername" to your actual GitHub username
```

2. `ecosystem.config.js` — Line 10

```
cwd: '/home/ubuntu/TRAININGBOT2',
// If your server username is different, update this path
```

3. `deploy.yml` — Lines 85-86

```
repo: 'git@github.com:yourusername/TRAININGBOT2.git'
path: '/home/ubuntu/TRAININGBOT2'
# Update to match your GitHub username and server path
```

4. GitHub Secrets (for auto-deploy)

Go to your GitHub repo → Settings → Secrets and variables → Actions → New repository secret:

Secret Name	Value	Needed For
`RENDER_SERVICE_ID`	Your Render service ID	Auto-deploy to Render
`RENDER_API_KEY`	Your Render API key	Auto-deploy to Render
`ORACLE_HOST`	Your Oracle VM IP address	Auto-deploy to Oracle
`ORACLE_SSH_KEY`	Your private SSH key	Auto-deploy to Oracle

🐋 Local Docker Test

Test everything locally before deploying:

```
cd C:\Users\fundi\OneDrive\Documents\TRAININGBOT2

# Build and run
docker-compose up -d

# Check logs
docker-compose logs -f neo-api

# Test API
curl http://localhost:8000/api/health

# Stop
docker-compose down
```

Generated by Kimi.ai

✅ Verification Checklist

- All 8 files copied to correct locations
- `wsgi.py` is in TRAININGBOT2 root
- `Dockerfile` is in TRAININGBOT2 root
- `.github\workflows\deploy.yml` exists
- `setup.sh` has your correct GitHub username
- `ecosystem.config.js` has correct server path
- `.env` file has all API keys (not committed to Git!)
- GitHub secrets added (if using auto-deploy)
- Tested locally with `docker-compose up`

🆘 Troubleshooting

Problem	Fix
`ModuleNotFoundError`	Run `pip install -r requirements.txt`
`wsgi:app not found`	Ensure `wsgi.py` is in TRAININGBOT2 root
Docker build fails	Check `Dockerfile` is in same folder as `launcher.py`
PM2 not found	Run `npm install -g pm2` on server
Port 8000 in use	Change `NEO_API_PORT` in `.env`

> **Questions?** The `deploy_to_trainingbot2.ps1` PowerShell script can auto-copy all files for you. Just run it in PowerShell!